04

PARTIE 4 • ANALYSE MODULAIRE

Les sous-programmes : fonctions et procédures

Ce que tu sauras faire à la fin

- Décomposer un problème complexe en sous-problèmes indépendants
- Choisir entre une **fonction** et une **procédure** selon le nombre de résultats
- Distinguer paramètre formel et paramètre effectif, objet local et objet global
- Choisir le bon mode de passage : par valeur ou par variable
- Écrire un programme principal qui n'est qu'une suite d'appels de modules

Pré-requis

- Les structures conditionnelles et itératives
- Les vecteurs et les chaînes
- La notion de variable

Mots-clés

module · fonction · def · return · procédure · paramètre · local · global

Durée conseillée : 5 séances

I Pourquoi découper un programme ?

Jusqu'ici nous avons écrit des programmes d'un seul bloc. Mais dès que le nombre de traitements augmente, le problème devient très complexe et difficile à résoudre d'un seul tenant.

Pour résoudre un problème de grande taille, on a intérêt à le **décomposer en sous-problèmes indépendants et de taille réduite**. À chaque sous-problème on associe un **module** qui assure sa résolution : ce module est soit une **fonction**, soit une **procédure**.

ASTUCE

Les quatre bénéfices d'une solution modulaire :
1. on résout un petit problème à la fois · **2.** on teste chaque module séparément · **3.** on réutilise un module dans un autre programme · **4.** le programme principal se lit comme un résumé de la solution.

II Les fonctions

DÉFINITION

Une **fonction** est un sous-programme qui résout un problème précis et qui doit avoir **un seul résultat de type simple** (entier, réel, booléen, caractère ou chaîne de caractères).

Exemples de traitements qui donnent une fonction

- Calculer la somme des entiers d'un tableau.
- Calculer la moyenne d'une classe.
- Déterminer le maximum d'un tableau d'entiers.
- Tester si un entier est premier (résultat booléen).

Vocabulaire et syntaxe

LANGAGE ALGORITHMIQUE	IMPLÉMENTATION EN PYTHON

`Fonction Nom_fonction (pf1: type1, … , pfn: typen) :`
`Type_résultat`
`DÉBUT`
`  <Traitement>`
`  Retourner Résultat`
`FIN`
	`def Nom_fonction(pf1, … , pfn) :`
	`  <Traitement>`
	`  return Résultat`

Caractéristiques

- Une fonction **retourne** une valeur d'un type identique à celui déclaré dans son en-tête.
- Elle comporte des paramètres **autres que le résultat**.
- C'est essentiellement un module de **calcul** qui renvoie une seule valeur d'un type simple.

REMARQUE

- Tout nouvel objet utilisé dans une fonction est un **objet local**.
- Tout objet déclaré dans le programme principal est un **objet global**.
- L'appel peut être effectué au niveau du programme principal ou d'un autre module appelant.
- Les **paramètres formels** figurent dans l'en-tête de la déclaration et correspondent à des variables locales utilisées dans le corps.
- Les **paramètres effectifs** sont utilisés dans l'instruction d'appel et sont substitués aux paramètres formels au moment de l'appel.
- Évite d'insérer des actions d'entrée / sortie dans une fonction.

ERREUR FRÉQUENTE

Une fonction qui affiche au lieu de retourner n'est plus réutilisable. `print` dans une fonction : presque toujours une erreur de conception. La règle : la fonction **calcule et retourne**, le programme appelant **affiche**.

ACTIVITÉ — 1 — COMPARER TROIS FONCTIONS

On veut déterminer le nombre de diviseurs d'un entier n strictement positif. Pour n = 6, le nombre de diviseurs est 4, car les diviseurs de 6 sont {1 ; 2 ; 3 ; 6}.
On te propose les trois fonctions suivantes :

NbDiv1	NbDiv2	NbDiv3

`Fonction NbDiv1(n:entier):entier`
`Début`
`  c ← 0`
`  Pour k de 1 à n faire`
`  Si (n mod k = 0) Alors`
`  c ← c+1`
`  Fin Si`
`  Fin Pour`
`  Retourner(c)`
`Fin`
	`Fonction NbDiv2(n:entier):entier`
	`Début`
	`  c ← 1`
	`  Pour k de 2 à (n div 2) faire`
	`  Si (n mod k = 0) Alors`
	`  c ← c+1`
	`  Fin Si`
	`  Fin Pour`
	`  Retourner(c)`
	`Fin`
		`Fonction NbDiv3(n:entier):entier`
		`Début`
		`  c ← 2`
		`  Pour k de 2 à (n-1) faire`
		`  Si (n mod k = 0) Alors`
		`  c ← c+1`
		`  Fin Si`
		`  Fin Pour`
		`  Retourner(c)`
		`Fin`

1) Complète le tableau par la valeur de c retournée par chaque fonction, pour **n = 4**.

Fonction	Valeur retournée
NbDiv1	*3*
NbDiv2	*2*
NbDiv3	*3*

2) Donne le nom de la fonction qui ne permet pas de retourner le nombre de diviseurs d'un entier n.

NbDiv2 : elle ne compte jamais n lui-même (n = 4 → 2 au lieu de 3 diviseurs). NbDiv3 est fausse seulement pour n = 1.

III Les procédures

DÉFINITION

Une **procédure** est un sous-programme qui résout un problème précis et qui peut avoir **zéro ou plusieurs résultats**.

LANGAGE ALGORITHMIQUE	IMPLÉMENTATION EN PYTHON

`Procédure Nom_procédure (pf1: type1, … , pfn: typen)`
`DÉBUT`
`  <Traitement>`
`FIN`
	`def Nom_procédure(pf1, … , pfn) :`
	`  <Traitement>`

1. Les types de paramètres

- Les **paramètres formels** figurent dans l'en-tête de la déclaration et correspondent à des variables locales utilisées dans le corps.
- Les **paramètres effectifs** sont utilisés dans l'instruction d'appel et sont substitués aux paramètres formels au moment de l'appel.

2. Les modes de passage des paramètres

La substitution des paramètres effectifs aux paramètres formels s'appelle **mode de passage de paramètres**. Il en existe deux :

Mode	Effet	Sens du transfert
Passage par valeur	Toute modification du paramètre formel n'a **aucun effet** sur le paramètre effectif.	Du programme appelant **vers** le programme appelé
Passage par variable (par référence, par adresse)	Toute modification portée sur le paramètre formel est **la même** sur le paramètre effectif.	Dans les **deux sens**

REMARQUE

- Avec les **fonctions** on utilise toujours le passage **par valeur**.
- Si le paramètre est **modifié** par la procédure (lecture ou affectation) → passage **par variable**.
- Si le paramètre n'est **pas modifié** (affichage, ou donnée utilisée pour un traitement) → passage **par valeur**.
- Si le mode de passage est par variable, on ajoute le symbole @ avant le nom du paramètre : `Procédure Saisir(@n : entier)`.

ASTUCE

En Python il n'y a pas de symbole @. Les **listes et les tableaux numpy sont modifiés sur place** (comportement équivalent au passage par variable), alors que les entiers, réels, booléens et chaînes ne le sont pas. Pour un entier modifié, on utilise soit `global`, soit — mieux — une fonction qui **retourne** la nouvelle valeur.

LES TROIS CAS À CONNAÎTRE

```python
# Passage par valeur : n n'est pas modifié à l'extérieur
def incr(n) :
    n = n + 1          # modifie seulement la copie locale

# Équivalent du passage par variable : on retourne la valeur
def saisir() :
    return int(input("n = "))

# Un tableau, lui, est bien modifié sur place
def remplir(n, T) :
    for i in range(n) :
        T[i] = int(input("T[" + str(i) + "] = "))
```

ACTIVITÉ — 2 — LES NOMBRES PARFAITS

On se propose d'écrire un programme qui permet de remplir un tableau T par n entiers strictement positifs (1 < n < 100), puis de déterminer et d'afficher les nombres parfaits qui figurent dans T.

Un nombre est dit parfait s'il est égal à la somme de ses diviseurs sauf lui-même.

Travail demandé : 1) écrire l'algorithme du programme principal ; 2) en déduire les algorithmes des modules ; 3) implémenter la solution en Python sous le nom `NbParfait`.

LANGAGE ALGORITHMIQUE

```
Algorithme NbParfait
Début
    Saisir(n)
    Remplir(n, T)
    Affiche(n, T)
Fin

T.D.N.T :          Type
    Tab = tableau de 100 entiers
T.D.O :
   Objet           Type / Nature
  n, T             entier, Tab
  Saisir           Procédure
  Remplir, Affiche Procédures
  Parfait          Fonction

Procédure Saisir(@n : entier)
Début
    Répéter  Ecrire("n = "), Lire(n)
    Jusqu'à (n > 1) ET (n < 100)
Fin

Procédure Remplir(n : Entier, @T : Tab)
Début
    Pour i de 0 à n-1 Faire
        Ecrire("T[", i, "]="), Lire(T[i]) ; Fin Pour
Fin

Fonction Parfait(m : Entier) : Booléen
Début
    S ← 0
    Pour i de 1 à m DIV 2 Faire
        Si m MOD i = 0 alors S ← S + i FinSi ; Fin Pour
    Retourner(S = m)
Fin

Procédure Affiche(n : Entier, T : Tab)
Début
    Pour i de 0 à n-1 Faire
        Si Parfait(T[i]) alors Ecrire(T[i]) FinSi ; Fin Pour
Fin
```

IMPLÉMENTATION EN PYTHON

```
from numpy import array
T = array([int] * 100)

def Saisir() :
    n = int(input("n = "))
    while not (1 < n < 100) :
        n = int(input("n = "))
    return n

def Remplir(n, T) :
    for i in range(n) :
        print("T[", i, "]=", end="")
        T[i] = int(input())

def Parfait(m) :
    S = 0
    for i in range(1, m // 2 + 1) :
        if m % i == 0 :
            S = S + i
    return S == m

def Affiche(n, T) :
    for i in range(n) :
        if Parfait(T[i]) :
            print(T[i])

# Programme principal
n = Saisir()
Remplir(n, T)
Affiche(n, T)
```

Fiche mémo — Les sous-programmes

Fonction	**un seul** résultat de type simple, obtenu par `Retourner` / `return`
Procédure	**zéro ou plusieurs** résultats, transmis par les paramètres
Paramètre formel	dans l'**en-tête** du module — c'est une variable locale
Paramètre effectif	dans l'**appel** — il remplace le paramètre formel
Passage par valeur	le module reçoit une copie : l'original n'est pas modifié
Passage par variable	noté @ : le module modifie l'original
Objet local / global	déclaré dans le module / déclaré dans le programme principal
Le bon réflexe	le programme principal ne doit contenir que des **appels**

SÉRIE D'EXERCICES 4

Les sous-programmes

6 exercices classés par difficulté · Facile Moyen Défi

Exercice 1 Tir à l'arc

Moyen

Un club de tir à l'arc organise une compétition. Chaque joueur dispose de **3 essais** de tir. À chaque tir, le joueur gagne de 0 à 5 points (un nombre **aléatoire**) selon l'endroit touché sur une cible composée de cinq cercles concentriques. Après le 3e essai, un **score** égal à la somme des trois essais est calculé pour chaque joueur.

On souhaite afficher les noms et les scores des n joueurs ayant participé à la compétition.

Travail demandé : écrire une solution **modulaire** — algorithme du programme principal, puis algorithme de chaque module envisagé.

EXEMPLE D'EXÉCUTION

```
n = 3
Noms : Amir | Nour | Sarra
Amir : 9   Nour : 12   Sarra : 7
```

Exercice 2 Bonus de recharge

Moyen

Pour son vingtième anniversaire, une société de télécommunication accorde un **bonus de recharge** à ses clients gagnants. Un client est gagnant si le montant de ses communications est **supérieur ou égal à 150 DT**.

Le programme doit :

- saisir le nombre de clients N (avec 5 < N < 100) ;
- remplir un tableau T par les numéros de téléphone des N clients, chaque numéro T[i] étant une chaîne de **huit chiffres** ;
- remplir un tableau C par les consommations téléphoniques, C[i] étant un entier positif exprimé en dinars ;
- afficher les numéros gagnants et leur bonus, obtenu en multipliant la consommation par 3.

Une solution modulaire est exigée.

EXEMPLE D'EXÉCUTION

```
Le numéro 66456789 a gagné 600 DT
Le numéro 65980765 a gagné 900 DT
Le numéro 66123321 a gagné 750 DT
Le numéro 65347743 a gagné 540 DT
```

Exercice 3 Cryptage d'un mot de passe

Défi

Pour accéder à sa session, un employé saisit son mot de passe. La sécurisation se fait grâce à une **clé de cryptage** selon le principe suivant :

- saisir le mot de passe MDP, composé uniquement de lettres majuscules et ne dépassant pas 10 caractères ;
- saisir un entier D dans l'intervalle [1 ; 5] ;
- saisir une clé CC, chaîne formée des chiffres 0 et 1 et de même longueur que MDP ;
- modifier chaque lettre de MDP d'indice i en ajoutant D à son code ASCII lorsque CC[i] = 1.

Exemple : MDP = "FGCTRUA", CC = "0011101", D = 2

Mot de passe	F	G	C	T	R	U	A
Clé de cryptage	0	0	1	1	1	0	1
Mot crypté	F	G	E	V	T	U	C

« C » devient « E » car sa valeur dans la clé est 1 et `chr(ord("C")+2) = "E"`.

Travail demandé : écrire l'algorithme du programme, décomposé en modules.

EXEMPLE D'EXÉCUTION

```
Le mot de passe après cryptage est : "FGEVTUC"
```

Exercice 4 CAPTCHA

Défi

Le CAPTCHA est une mesure de sécurité de type authentification par question-réponse, qui vérifie que l'utilisateur est bien un être humain. Ici on considère les CAPTCHA basés sur des opérations de calcul.

On utilisera deux tableaux : T1 contiendra des opérations et T2 leurs résultats correspondants.

- chaque élément T1[i] contient une opération de la forme `"a * b = ?"`, a et b étant deux nombres entre 0 et 9 générés au hasard ;
- T2[i] contient le résultat de l'opération T1[i] ;
- le programme affiche **aléatoirement** une opération de T1 ;
- l'utilisateur saisit sa réponse ;
- le programme affiche « Authentification valide » si la réponse est correcte, « Authentification invalide » sinon.

Travail demandé : écrire l'algorithme du programme principal en le décomposant en modules, puis l'algorithme de chaque module.

EXEMPLE D'EXÉCUTION

```
N = 6
T1 : "1*6=?" | "4*5=?" | "2*8=?" | "3*6=?" | "8*1=?" | "2*9=?"
T2 : 6 | 20 | 16 | 18 | 8 | 18
Si le programme choisit T1[4], il affiche "8*1=?"
Réponse 8 → Authentification valide
```

Exercice 5 Majuscules

Moyen

Écrire une solution modulaire qui permet de saisir une chaîne de caractères CH, puis de construire et d'afficher la chaîne obtenue en convertissant en majuscules toutes les lettres minuscules de CH, **sans utiliser la fonction prédéfinie `upper()`**.

On utilisera une fonction `Majus(c)` qui convertit un caractère, et une fonction `Convertir(ch)` qui traite la chaîne entière.

EXEMPLE D'EXÉCUTION

```
CH : "bac 2026 SI"
BAC 2026 SI
```

Exercice 6 Classement de chansons — Hitparade

Défi

Une station de radio lance chaque semaine un concours intitulé « Hitparade » pour classer cinq titres de chansons. Chaque auditeur choisit son titre préféré, ce qui ajoute **3 points** au score de ce titre.

Le programme doit :

- remplir un tableau T1 par les cinq titres (uniquement des lettres alphabétiques et des espaces) ;
- saisir le nombre n de participants (5 < n < 100) ;
- générer un tableau T2 représentant les scores, en ajoutant 3 points au score d'une chanson chaque fois que son numéro (de 0 à 4) est choisi ;
- afficher le classement en commençant par le plus grand score. **Les chansons à score égal ont le même rang.**

Travail demandé : 1) algorithme du programme principal décomposé en modules ; 2) algorithmes des modules ; 3) implémentation en Python.

EXEMPLE D'EXÉCUTION

```
T1 : Happy | Sorry | Me Quemo | Rosa | Hello  ·  n = 11
T2 : 9 | 6 | 0 | 6 | 12
Rang 1 : Hello
Rang 2 : Happy
Rang 3 : Sorry
Rang 3 : Rosa
Rang 5 : Me Quemo
```